

问题整理

目前的PPO设计：

1. 状态表示
2. 动作表示
3. 策略网络结构
4. 环境交互

目前存在的问题：

目前的PPO设计：

1. 状态表示

- 物理环境：
 - 每个物理节点的CPU总量、CPU使用率、内存总量、内存使用率
 - 每个物理链路的带宽总量和已使用量
 - 物理网络中的边
- 虚拟工作：
 - 每个任务节点资源需求（CPU、内存）
 - 每条任务链路有其带宽需求范围（min_bw, max_bw）
 - 虚拟网络中的边

2. 动作表示

- 任务分配（离散）：一个整数向量，长度等于虚拟任务数，每个元素表示该虚拟任务被分配到的物理节点ID。
- 带宽分配（离散）：一个整数，长度等于虚拟链路数，每个元素表示给该链路分配的等级（将 [min_bw, max_bw] 离散成10个等级）。

3. 策略网络结构

- 映射策略网络（MappingActor）
 - 编码：两套 GraphEncoder(GATConv) 分别编码物理/虚拟图；可选融合 GCN 特征（线性投影

后与编码相加)。

- 融合: MultiheadAttention 做 虚拟→物理 的注意力, 将注意后的虚拟表示与原虚拟编码拼接。
- 策略头: MLP 输出每个虚拟节点映射到物理节点的 logits (动态裁剪到实际物理节点数)。
- 约束头: MLP 输出每个虚拟节点的可行性分数, 再广播到各物理节点, 作为对数加权项。
- BandwidthActor: 两个 GAT 编码器 + 链路 MLP 编码器 (基于虚拟边两端节点特征拼接) + MultiheadAttention (链路→映射后的物理编码) + MLP 策略头 (带宽等级 logits) + 约束 MLP。
- 带宽策略网络 (BandwidthActor)
 - 编码: 两套 GraphEncoder(GATConv) 编码物理/虚拟图; 可选融合 GCN 特征。
 - 链路建模: 对每条虚拟边拼接两端节点特征, 经 link_encoder 得到链路表示。
 - 融合: 用 MultiheadAttention 以链路为查询, 注意已映射的物理节点编码; 再与全局物理/虚拟均值、同节点映射特征拼接。
 - 策略头: MLP 输出每条链路的带宽等级 logits; 同节点映射的链路对最大带宽级别有增强。
 - 约束头: MLP 输出链路可行性分数, 作为对数加权项。
- Critic
 - 两套图编码器 (GAT) 分别编码物理图与虚拟图, 做全局平均池化后拼接, 再经 MLP 回归一个标量价值; 可选融合外部 GCN 特征 (线性投影后与编码相加), 不使用多头注意力。

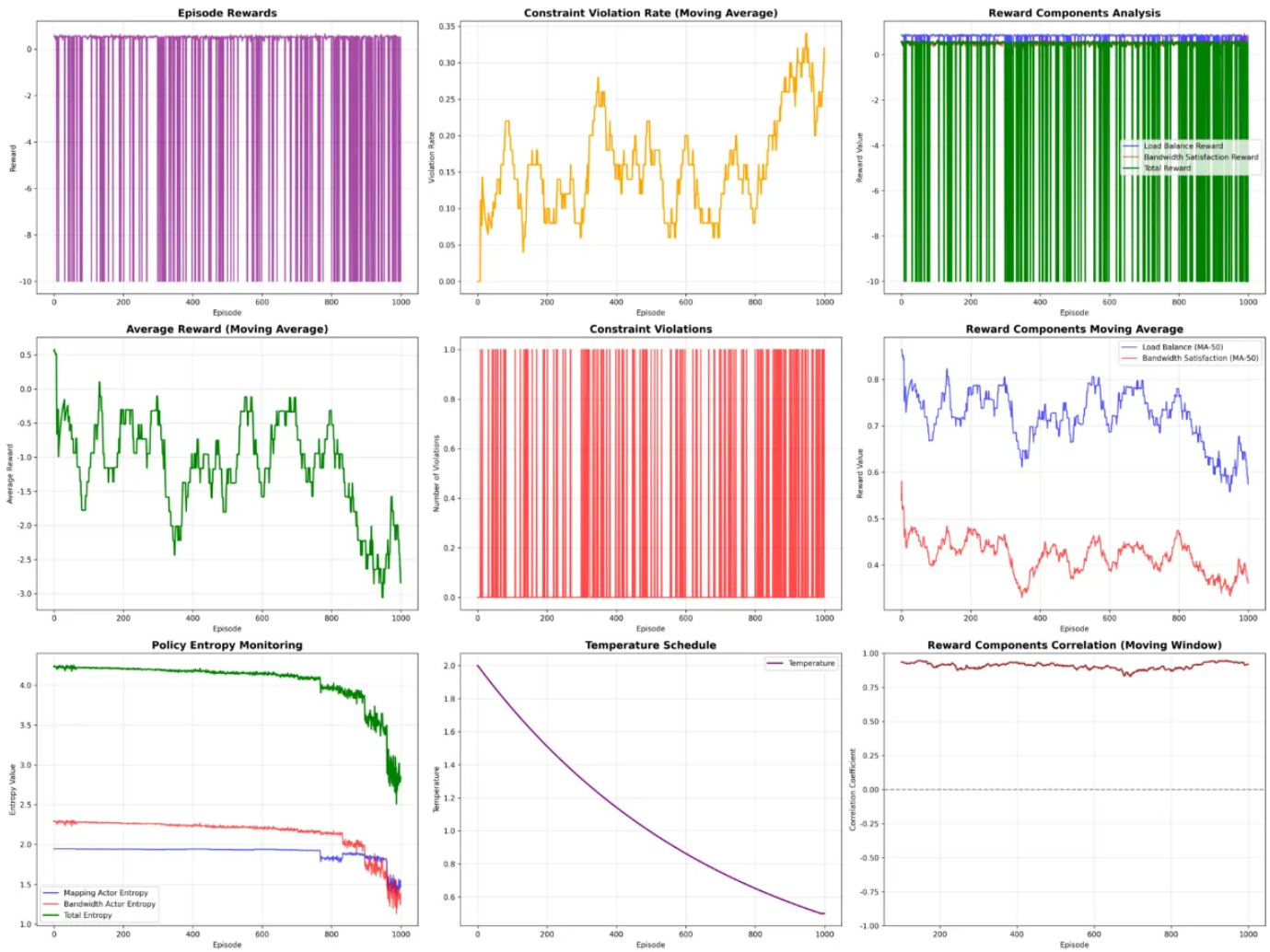
4. 环境交互

- 初始化环境: 生成一个物理环境和一个虚拟工作。
- 每个episode: 智能体对当前状态做出决策 (任务分配和带宽分配)。(只有一个step)
- 环境应用决策, 检查资源约束, 如果违反则给予惩罚 (-10), 并计算负载均衡指标和带宽满足度, 假如负载过高 (>0.85) 给予惩罚, 然后计算奖励。
- 动作选择时 (推理) 与训练更新时 (每个 epoch 的前向重算), 先用硬约束屏蔽不可行动作 (logits 置 -inf, 再温度缩放), 随后叠加软约束打分的对数项以引导偏好。
 - 硬约束 (ConstraintManager): 不可行动作直接屏蔽 (logits 置为 -inf), 带温度缩放。(如资源不够)
 - 软约束 (Actor 内部 MLP): 对可行性打分, 作为对数加权项加入 logits。(同节点映射链路的带宽 logits 提升/采样偏置 (偏向最大带宽))

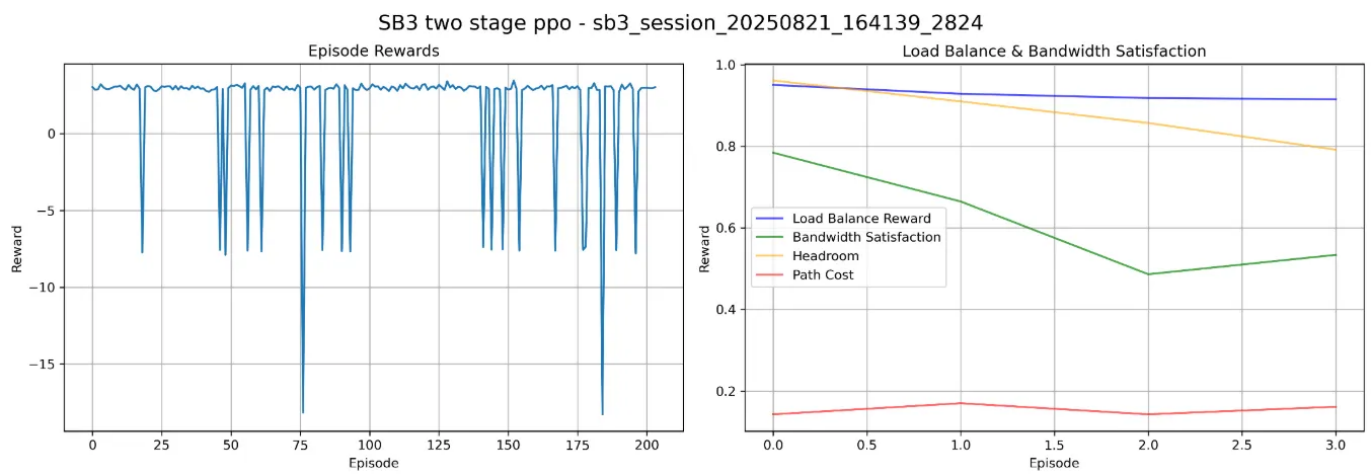
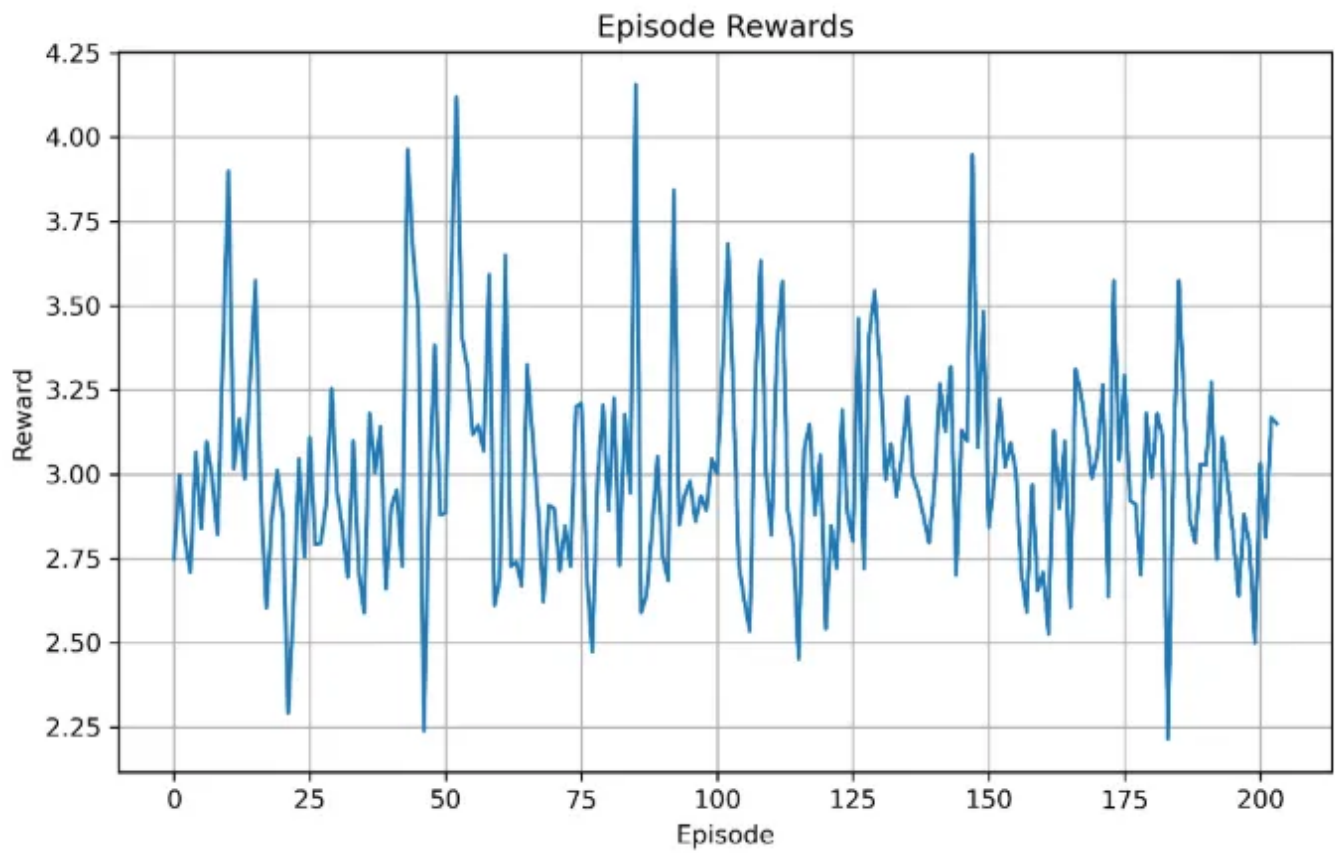
目前存在的问题:

算法不收敛，智能体学不到东西，用SB3写了同样的算法的简单实现（没有用到GAT），也是不收敛。

然后在SB3的实现上修改环境为：每个episode初始化一个物理网络环境，初始负载均为0，每个step处理一个虚拟工作，每个episode处理5个虚拟工作，每个step接收虚拟工作之后得到一个节点映射和带宽分配的结果，并更新物理网络环境。也还是不收敛。



自己写的PPO



sb3实现的新环境